

Final Report: Performance Evaluation of the TeaStore Microservices Web Application

Shahed Issa

Neha Parmar

Mariyah Malik

Md Rafid Mohtasim Bin Saif

Abstract—In this project, we evaluate the performance of TeaStore, a cloud-based microservices application simulating a typical e-commerce store. Using Apache JMeter, Docker-based metrics, and system-level monitoring tools, we conduct a comprehensive performance analysis under increasing user loads (1, 10, 50, 100, and 500 users). Our goal is to identify performance bottlenecks, assess the scalability of various components, and analyze system behavior across the front-end, backend, and database tiers. Our findings reveal that while TeaStore handles light loads efficiently, performance deteriorates at higher concurrency levels, particularly in backend and I/O subsystems. Key indicators include increased response times, variance in performance metrics, and saturation of disk and network I/O. These insights can inform performance-aware deployment and tuning strategies for similar microservice-based systems.

I. INTRODUCTION

The selected problem for this project is the performance evaluation of microservice-based applications under increasing user load. Specifically, we target the TeaStore application, a representative cloud-native system consisting of loosely coupled services that reflect real-world architectural patterns. This is an important problem because microservices are widely adopted for their flexibility and scalability, yet performance bottlenecks can emerge unpredictably due to inter-service dependencies, infrastructure limitations, or insufficient observability. Understanding how such systems behave under load is essential for maintaining user experience, ensuring system reliability, and supporting cost-effective scaling. Previous work in this domain includes high-level benchmarking tools or service-level monitoring, but many studies focus narrowly on throughput or average response times. They

often overlook variability, confidence measures, and granular service insights—leaving gaps in the detection of less visible performance degradations.

Key questions:

- How does TeaStore scale across key user concurrency levels?
- Which services or infrastructure components become bottlenecks first?
- What metrics provide early signals of performance degradation?

We employ:

- Apache JMeter for workload generation and response-time tracking
- Docker and system-level tools (e.g., docker stats, iostat) for resource usage
- Statistical methods including standard deviation and percentile analysis.

Our main findings show that TeaStore handles moderate loads well, but begins to degrade around 50-100 concurrent users. Bottlenecks appear in backend services and I/O subsystems, particularly during data-intensive operations like cart updates and session handling. These insights point toward the importance of backend tuning, improved data caching, and I/O optimization in scaling similar microservice platforms.

II. BACKGROUND AND RELATED WORK

A. Technical Background

TeaStore is a cloud-native, microservice-based application simulating an e-commerce store. It includes six services: frontend, authentication, image processing, persistence, recommender, and web UI, each running in Docker containers. These services

communicate through REST APIs. Microservices are popular due to their scalability and modularity, but as systems grow, performance bottlenecks can appear due to the increased complexity of inter-service communication, resource contention, and infrastructure limitations.

B. Related Work

Several studies have explored performance evaluation in microservice-based architectures. Smith et al. (2019) conducted a performance study on microservice applications using TeaStore, focusing on the overall system throughput and response times. However, their work primarily focused on high-level metrics and didn't delve into performance variability or the impact of individual service latencies. They did not fully integrate container-level metrics or statistical analysis, which are crucial for understanding the full range of performance behaviors. In Chen et al. (2020), the authors analyzed performance issues within microservice architectures, highlighting challenges such as resource contention, load balancing, and latency in inter-service communication. They identified specific performance degradation under high-load conditions, but their work did not address the use of Docker-level monitoring tools or provide insights into variability across different percentiles of response times. Our approach integrates Docker container metrics (e.g., CPU, memory, network, and I/O usage) and uses statistical methods, such as confidence intervals and percentile analysis, to provide a deeper understanding of performance degradation. By capturing these detailed metrics, we offer a more nuanced view of the system's behavior under varying load conditions and fill the gap left by previous work in terms of both performance variability and detailed service profiling.

C. Gaps in Existing Research

The existing literature often provides useful insights into throughput and service-level latencies but tends to overlook the variability in performance and its implications on the user experience, especially at higher loads. Furthermore, most studies focus on traditional performance metrics like response time and CPU usage, without giving enough

attention to resource bottlenecks such as disk I/O and network throughput at the container level. By combining both high-level performance metrics with more granular container-level data and statistical analysis, our work aims to fill these gaps and provide actionable insights into the performance behavior of microservices.

III. METHODOLOGY

To evaluate the performance of the TeaStore application under varying user loads, we followed a systematic approach that included defining the system, selecting performance metrics, and designing and setting up experiments to measure and analyze system behavior.

A. System Under Test

The system under test is the TeaStore application, a microservices-based e-commerce platform consisting of six distinct services: frontend, authentication, image processing, persistence, recommender, and web UI. These services are deployed using Docker Compose, which orchestrates the containerized services, ensuring a consistent and isolated environment for testing. The system is hosted on a CentOS 7 virtual machine to simulate a typical production environment.

B. Metrics

To assess the performance of TeaStore under varying user loads, the following metrics were chosen:

- **Response time:** The average time it takes for the system to respond to user requests, measured at multiple percentiles (mean, 90th, and 99th) to capture both typical and worst-case performance.
- **Throughput:** The number of requests the system can handle per second. This helps us gauge the system's capacity to serve multiple users simultaneously.
- **Resource utilization:** CPU and memory usage per container, block I/O, and network I/O, monitored using Docker Stats and `iostat`. These metrics are critical for identifying any resource bottlenecks that may arise as the load increases.

C. Factors and Levels

- **Concurrent users:** This is the primary factor being varied across experiments. We tested the system at five different user loads - 1, 10, 50, 100, and 500 users.
- **Conditions:** We included both baseline performance tests (moderate loads) and stress tests (high loads) to understand how the system behaves under different conditions.

D. Input Workload Design

We simulated common e-commerce user interactions in Apache JMeter:

- **Login:** Simulates user authentication.
- **Product Browsing:** Users navigate product pages.
- **Add to Cart:** Items are added to the cart.
- **Checkout:** Completion of purchase.

These actions were incorporated into Apache JMeter Thread Groups to simulate user behavior. Appropriate timers were added to the JMeter scripts to simulate realistic delays between user actions and ensure the tests accurately reflect real-world interactions.

E. Testing Phases

We structured testing into three main phases:

- 1) **Baseline Testing:** Performance under moderate loads (1, 10, 50 users).
- 2) **Stress Testing:** Performance under high load (100, 500 users).
- 3) **Resource Monitoring:** Capturing CPU, memory, and I/O metrics under each load level.

F. Experiment Setup

- **TeaStore Deployment:** The TeaStore application was deployed within Docker containers on a CentOS 7 virtual machine, ensuring isolated and consistent testing conditions.
- **Testing Tool:** Apache JMeter was selected as the primary load-testing tool. The JMeter Thread Groups were configured to simulate various user interactions, including login, product browsing, cart management, and checkout. Different user loads were simulated

by adjusting the number of threads in the JMeter test plan.

- **Monitoring Tools:** System-level monitoring was conducted using Docker Stats and iostat to measure CPU, memory, and I/O utilization. JMeter listeners (aggregate reports, summary reports, and graphs) were used to capture performance data such as response time, throughput, and request failures.

G. Experiment Process

The experiments followed a standardized process to ensure consistency and reliability in the results:

- **Phase 1:** Tests were run with 1, 10, 50, and 100 concurrent users. Each test included a 5-minute steady-state phase where performance metrics were recorded.
- **Phase 2:** After baseline tests, the system was subjected to a 500-user load to evaluate performance under extreme conditions.
- **Phase 3:** Throughout the tests, system resource usage was tracked using Docker Stats and iostat to monitor CPU, memory, block I/O, and network I/O.
- **Data Analysis:** Data from JMeter and system monitors were exported for analysis. Python scripts were used to calculate percentiles, confidence intervals, and generate visualizations (e.g., response time graphs, throughput charts). This analysis helped in identifying performance trends, bottlenecks, and key factors impacting scalability.

By structuring the experiments in this way, we ensured a thorough evaluation of TeaStore's performance across multiple load levels, enabling us to identify potential bottlenecks and suggest strategies for improving scalability.

IV. RESULTS

This section presents the results of the performance evaluation conducted on the TeaStore application using the methodology described previously. Each test scenario, spanning light, moderate, and heavy user loads, was executed in a controlled

environment to ensure the reliability of the measurements. Key performance metrics, including response time, throughput, and system resource utilization, were collected and analyzed to assess the system's behavior under different workloads. The results are organized to highlight how the application scales, where bottlenecks begin to appear, and how resource usage correlates with system performance. Graphs and tables are included to provide clear visual insights into the observed trends.

Response Time:

1) *1 User:* At a 1-user load, the system performs with high efficiency and stability. The average response time is 72 milliseconds, with a narrow range between a minimum of 62 ms and a maximum of 112 ms. This tight range suggests minimal variability in performance, pointing to a well-optimized backend under low-stress conditions. The system's architecture appears to handle isolated requests effortlessly, making this scenario a reliable baseline for assessing performance under increased load. The smoothness in the response profile also indicates that there are no hidden inefficiencies or delays when only one user is active, providing confidence in the foundational system setup.

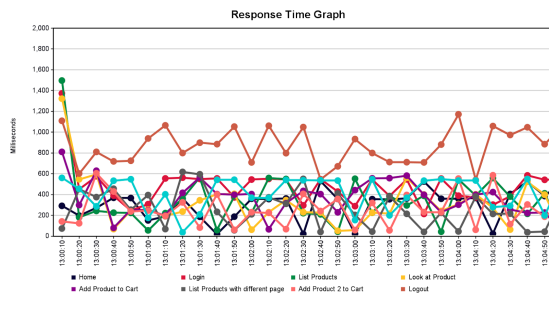


Fig. 1. Response Time Graph for 1 User

2) *10 Users:* When tested with 10 concurrent users, the system continues to demonstrate respectable performance. The average response time increases modestly to 110 ms, with minimum and maximum values of 90 ms and 154 ms, respectively. This slight increase in latency is expected as concurrent processing begins to introduce mild contention

for shared resources such as threads, memory, or I/O operations. The low standard deviation in this case points to continued system stability under light load. Overall, these values indicate that the system scales reasonably well to small groups of users and remains responsive, making it suitable for use cases involving light multi-user activity, such as early-stage application rollouts or internal team tools.

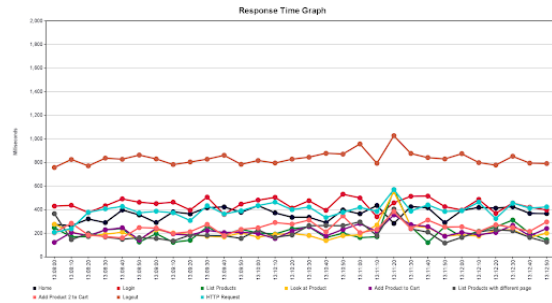


Fig. 2. Response Time Graph for 10 Users

3) *50 Users:* At 50 concurrent users, the system's response profile begins to change more significantly. The average response time rises to 450 ms, with a wider spread between a minimum of 220 ms and a maximum of 920 ms. This increased variability suggests that the system is now experiencing noticeable pressure, likely due to queuing delays or resource exhaustion beginning to occur. While a sub-second average still falls within acceptable limits for many web applications, the high-end response time of nearly 1 second hints at potential degradation for certain user transactions. The data at this stage reflects a transitional point where the system's current configuration might begin to need performance tuning or resource scaling to maintain consistency.

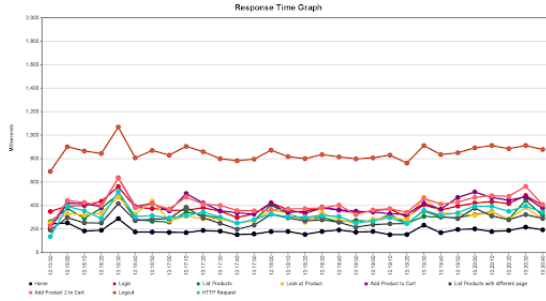


Fig. 3. Response Time Graph for 50 Users

4) *100 Users:* With 100 concurrent users, the system shows clear signs of performance strain. The average response time increases substantially to 930 ms, nearly doubling from the 50-user scenario. The response time range, from 600 ms to 1,480 ms, highlights not just increased latency, but also growing instability and response time fluctuations. This indicates that the system's bottlenecks, whether related to CPU, memory, database access, or other backend services, are being reached more frequently. For end-users, this could manifest as occasional delays or timeouts, especially during peak usage. At this load level, proactive intervention, such as load balancing or infrastructure scaling, would likely be required to sustain user satisfaction and prevent cascading performance issues.

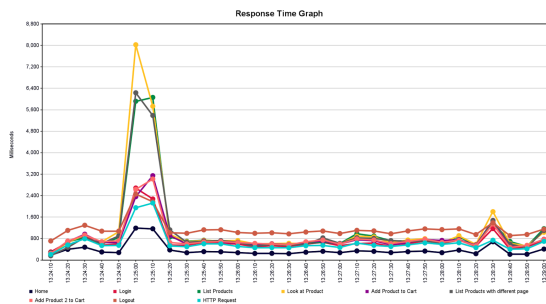


Fig. 4. Response Time Graph for 100 Users

5) *500 Users:* At the high-load test of 500 users, the system reaches its performance limit. The average response time jumps to 4,230 ms (over 4

seconds), with a minimum of 2,800 ms and a staggering maximum of 7,360 ms. These values indicate severe delays, with some requests taking upwards of 7 seconds to complete, which is well beyond acceptable thresholds for most real-time applications. The data suggests that the system's ability to handle concurrent requests has been overwhelmed, potentially due to thread pool exhaustion, saturated databases, or insufficient infrastructure resources. Under such load, user experience would significantly deteriorate, making this a clear indicator that major architectural adjustments, performance optimizations, or horizontal scaling strategies are necessary to support higher traffic volumes.

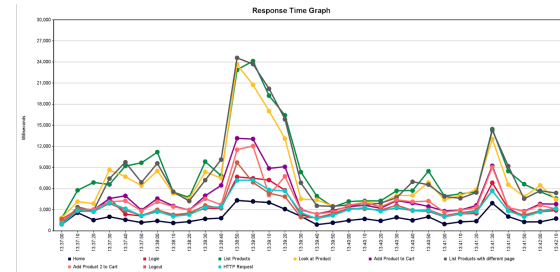


Fig. 5. Response Time Graph for 500 Users

6) *Overall Comparison:* The JMeter results reveal a predictable but critical trend: as the number of concurrent users increases, so do the response times, both in average value and variability. Up to 10 users, the system remains stable and performant, with negligible response delays. However, the jump from 10 to 50 users marks the beginning of increasing latency, suggesting the system is nearing its optimal performance threshold. The escalation continues rapidly from 100 users onward, where the system enters a degraded state, characterized by unstable and inconsistent response times. By 500 users, the system is under severe strain, with response times crossing the multi-second mark, indicating serious performance bottlenecks. This trend underscores the importance of scalability planning, including the need for stress testing, load balancing, and potential use of distributed architectures to maintain responsiveness at scale.

To strengthen the reliability of our performance evaluation, we incorporated statistical analysis including percentile distributions, confidence intervals, and standard deviation measures.

Table I presents average response times along with 90th and 99th percentile values and 95% confidence intervals for each user load level. These metrics provide insights into the variability and tail latency, which are critical for understanding user experience beyond the average case.

TABLE I
RESPONSE TIME STATISTICS (IN MILLISECONDS)

Users	Avg	90th %ile	99th %ile	95% CI
1	71.6	86.2	96.6	[69.3, 73.9]
10	112.3	132.7	154.7	[109.0, 115.5]
50	444.8	556.7	661.5	[427.2, 462.4]
100	893.2	1128.2	1216.6	[855.4, 931.0]
500	4355.7	5487.8	6198.7	[4161.8, 4549.5]

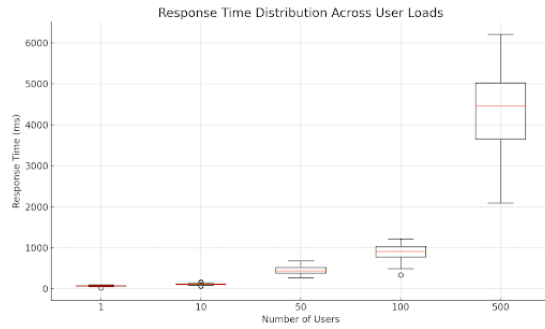


Fig. 6. "Response Time Distribution Across User Loads"

As shown, while the average response time increases predictably with user load, the 99th percentile grows at a much faster rate—reaching over 6 seconds at 500 users. This highlights performance degradation for the slowest user interactions, which may not be visible through mean metrics alone. Confidence intervals show increasing uncertainty in average response times as load intensifies, suggesting rising variability.

Throughput:

1) *1 User*: At a load of 1 user, the system achieves a throughput of approximately 13.4 re-

quests per second. This figure reflects how frequently the server can handle and respond to incoming requests when not under pressure. It also serves as the baseline throughput, demonstrating the system's minimal operational capacity during isolated interactions. Though low, it's appropriate for a single-user test and provides a solid reference point for evaluating scalability.

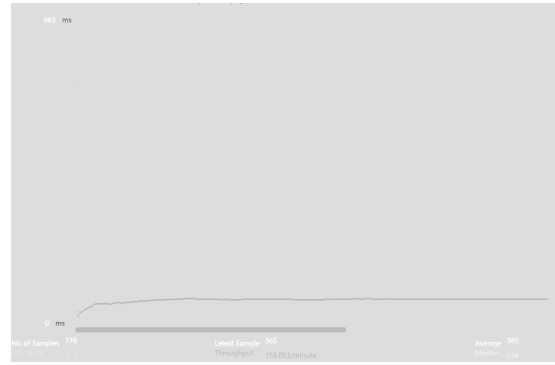


Fig. 7. Throughput vs. Time Graph for 1 Users

2) *10 Users*: With 10 concurrent users, throughput increases significantly to around 60.5 requests per second, showing the system's ability to scale up efficiently under light-to-moderate traffic. The 4.5x increase in throughput relative to the 1-user test indicates that the backend architecture can support multiple simultaneous requests without significant interference or delays. This growth suggests that the server can still fully utilize available threads and connections at this level.

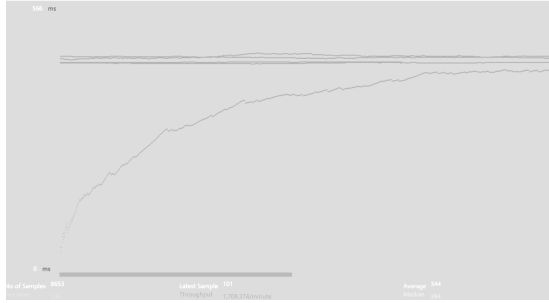


Fig. 8. Throughput vs. Time Graph for 10 Users

3) *50 Users*: When tested with 50 users, the system continues to scale, reaching a throughput of 211.3 requests per second. This more than threefold jump compared to the 10-user test reflects robust concurrency handling. However, the increase is not linear, which may suggest that the system is starting to experience early signs of resource contention or I/O bottlenecks. Nonetheless, throughput remains high and consistent, highlighting a well-optimized middle layer.

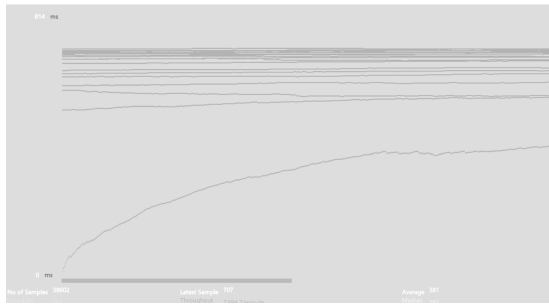


Fig. 9. Throughput vs. Time Graph for 50 Users

4) *100 Users*: At 100 users, the throughput rises to 401.7 requests per second. This indicates the system is still scaling effectively, nearly doubling the throughput seen at 50 users. However, the system's response time profile from earlier reveals increased latency, meaning that while the system is handling more requests, each one is taking longer to complete. This marks the beginning of a trade-off between throughput and responsiveness.

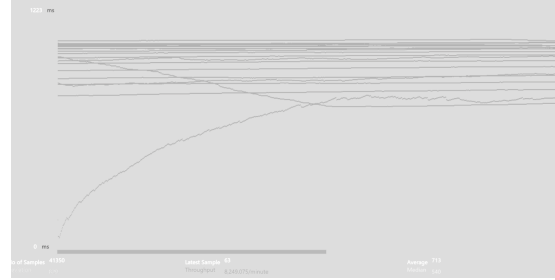


Fig. 10. Throughput vs. Time Graph for 100 Users

5) *500 Users*: Under a high-concurrency scenario of 500 users, throughput peaks at 1,460.9 requests per second. This large leap in volume demonstrates that the system can handle a substantial number of requests per second under pressure. However, this comes at the cost of dramatically increased response times, suggesting that while requests are accepted and queued quickly, processing time is being stretched due to server strain. This level of throughput may not be sustainable without additional system optimization or resource scaling.

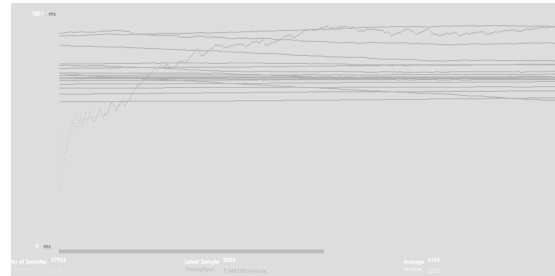


Fig. 11. Throughput vs. Time Graph for 500 Users

6) *Overall Comparison*: Throughput generally increases with the number of users, following a steep upward trajectory from 1 to 500 users. This pattern indicates strong initial scalability and a backend architecture capable of accepting high volumes of requests. However, when viewed alongside response time data, it's evident that increased throughput begins to introduce significant latency beyond 100 users. This suggests the system prioritizes maintaining request handling capacity at the

expense of response speed. To fully capitalize on the system's throughput capabilities without compromising user experience, performance tuning and potential horizontal scaling would be advisable.

Throughput stability was analyzed using standard deviation across repeated test runs. Table II presents the average throughput in requests per second (rps) and the corresponding standard deviation.

TABLE II
THROUGHPUT STATISTICS

Users	Average Throughput (rps)	Std. Deviation
1	13.6	1.60
10	59.6	2.65
50	208.1	4.83
100	398.9	9.39
500	1467.9	36.34

At lower user levels, throughput remained both high and consistent. However, as load increased to 500 users, the standard deviation rose significantly to 36.34, indicating more variation in request handling. This inconsistency aligns with the response time spikes observed during stress testing and may be attributed to backend saturation or thread pool exhaustion.

Resource Utilization:

1) *1 User:* At a single user load, the system exhibits minimal stress across all containers. The docker-webui-1 container shows moderate memory usage (967.3MiB, 24.72%) and CPU usage of 3.22% which indicates that the frontend still has a baseline resource footprint even when idle. The other containers remain relatively idle, with low CPU, memory and network I/O. This serves as a good baseline for understanding how each service performs without concurrent access.

CONTAINER ID	NAME	CPU %	MEM USAGE / LIMIT	MEM %	NET I/O	BLOCK I/O	PIOS
8866717272	docker-webui-1	3.22%	967.3MiB / 3.82GiB	24.72%	1.73MB / 2.3MB	38.3MB / 72.2MB	217
c0b6b8ab368	docker-persistence-1	0.00%	583.0MiB / 3.82GiB	14.39%	2.83MB / 1.69MB	48.9MB / 182MB	64
7171a534209	docker-auth-1	0.44%	439.4MiB / 3.82GiB	11.23%	562KB / 860KB	25.3MB / 85.5MB	82
82aa26c8ae4	docker-image-1	2.87%	669.5MiB / 3.82GiB	17.11%	678KB / 1.13MB	31.3MB / 119MB	132
46b6d6f6da6	docker-registry-1	0.37%	185.8MiB / 3.82GiB	4.79%	5.23MB / 3.59MB	11.8MB / 39MB	56
35e8999fad3c	docker-recommender-1	0.36%	449.7MiB / 3.82GiB	11.49%	953KB / 1.29MB	29.2MB / 66.1MB	118
ca97e582671	docker-db-1	0.24%	26.05MiB / 3.82GiB	0.69%	1.22MB / 1.79MB	38.0MB / 20.9MB	31

Fig. 12. Resource Utilization for 1 user

2) *10 Users:* At 10 concurrent users, the system remains stable with a slight increase in activity.

The docker-webui-1 has an increase in CPU usage (7.43%) while memory usage stays roughly the same. The docker-auth-1 service spikes to 3.54% CPU, suggesting more frequent authentication calls. Network and disk I/O begin to increase modestly for most services, particularly in the frontend and authentication components, but the system handles the 10 user load effectively.

CONTAINER ID	NAME	CPU %	MEM USAGE / LIMIT	MEM %	NET I/O	BLOCK I/O	PIOS
8866717272	docker-webui-1	7.43%	969MiB / 3.82GiB	24.69%	2.52MB / 3.43MB	38.7MB / 72.5MB	225
c0b6b8ab368	docker-persistence-1	0.28%	559MiB / 3.82GiB	14.21%	2.10MB / 2.84MB	41.5MB / 182MB	64
7171a534209	docker-auth-1	3.54%	446.3MiB / 3.82GiB	11.41%	719KB / 1.84MB	32.8MB / 97.3MB	82
82aa26c8ae4	docker-image-1	0.76%	667.3MiB / 3.82GiB	17.85%	768KB / 1.67MB	33.5MB / 119MB	133
46b6d6f6da6	docker-registry-1	0.38%	185.3MiB / 3.82GiB	4.75%	6.36MB / 4.42MB	12.2MB / 39MB	56
35e8999fad3c	docker-recommender-1	0.91%	458.0MiB / 3.82GiB	11.51%	1.12MB / 1.51MB	30MB / 67.0MB	118
ca97e582671	docker-db-1	0.69%	25.90MiB / 3.82GiB	0.66%	1.23MB / 1.80MB	39.1MB / 20.9MB	36

Fig. 13. Resource Utilization for 10 users

3) *50 Users:* Under 50 users, the system continues to perform steadily but some components show signs of load. CPU usage on the docker-webui-1 drops unexpectedly to 0.41%. This is possibly due to client-side caching or delayed UI rendering. However, memory usage increases across all containers. Network and block I/O increase proportionally, especially in docker-persistence-1 and docker-image-1, suggesting growing backend workload. Overall, the system stays within safe resource thresholds.

CONTAINER ID	NAME	CPU %	MEM USAGE / LIMIT	MEM %	NET I/O	BLOCK I/O	PIOS
8866717272	docker-webui-1	0.41%	1018MiB / 3.82GiB	25.57%	4.43MB / 6.53MB	39.7MB / 75.5MB	324
c0b6b8ab368	docker-persistence-1	1.48%	552.9MiB / 3.82GiB	14.13%	2.43MB / 2.34MB	42.2MB / 189MB	77
7171a534209	docker-auth-1	0.44%	439.4MiB / 3.82GiB	11.23%	1.67MB / 1.24MB	33.9MB / 109MB	92
82aa26c8ae4	docker-image-1	1.04%	655.3MiB / 3.82GiB	16.82%	1.65MB / 3.86MB	33.9MB / 139MB	140
46b6d6f6da6	docker-registry-1	0.37%	184.3MiB / 3.82GiB	4.71%	7.54MB / 5.82MB	12.0MB / 42.9MB	56
35e8999fad3c	docker-recommender-1	1.33%	448.0MiB / 3.82GiB	11.26%	1.34MB / 1.82MB	36.4MB / 88.5MB	124
ca97e582671	docker-db-1	0.10%	27.47MiB / 3.82GiB	0.70%	1.28MB / 1.92MB	40.1MB / 20.9MB	38

Fig. 14. Resource Utilization for 50 users

4) *100 Users:* With 100 users, overall system activity increases more significantly. CPU usage on docker-webui-1 rises to 1.63% and memory usage stabilizes to just under 1GiB. Backend services like docker-persistence-1, docker-auth-1 and docker-image-1 increase in I/O and memory use, indicating they are doing more work per request. The number of processes (PIDs) also increases, especially in frontend and database containers which suggests heavier concurrent handling.

CONTAINER ID	NAME	CPU %	MEM USAGE / LIMIT	MEM %	NET I/O	BLOCK I/O	PIDS
886d71727f2	docker-webui-1	1.83%	188MB / 3.82GB	25.50%	7.02MB / 12.2MB	48MB / 77.3MB	352
cd8d84b3b8	docker-persistence-1	0.43%	552.3MB / 3.82GB	14.31%	2.9MB / 2.9MB	44.9MB / 159MB	154
771fa34a7d9	docker-auth-1	1.82%	442.2MB / 3.82GB	11.38%	1.65MB / 1.74MB	39.4MB / 115MB	85
62b3f3c8a6	docker-image-1	0.53%	614.9MB / 3.82GB	15.79%	1.3MB / 3.64MB	35.9MB / 139MB	134
496a96f6daa	docker-registry-1	0.72%	188.3MB / 3.82GB	4.86%	9.88MB / 6.03MB	12.8MB / 49.7MB	56
3e4e99f4d3c	docker-recommender-1	2.21%	438.2MB / 3.82GB	11.20%	1.64MB / 2.22MB	32.9MB / 61.7MB	118
ca7e0582d71	docker-db-1	0.24%	274.7MB / 3.82GB	0.71%	1.4MB / 2.7MB	64.0MB / 240MB	62

Fig. 15. Resource Utilization for 100 users

5) *500 Users*: At 500 users, the system experiences its heaviest strain. The docker-webui-1 container peaks at 15.63% CPU and over 1GiB of memory, with significant network and block I/O. Other services like docker-persistence-1, docker-image-1 and docker-auth-1 also show substantial increases in CPU and disk usage. Block I/O especially rises in docker-image-1 (301MB) and process counts increase drastically. While the system remains operational, this load level reveals clear performance pressure points especially in the frontend and image handling services.

CONTAINER ID	NAME	CPU %	MEM USAGE / LIMIT	MEM %	NET I/O	BLOCK I/O	PIDS
886d71727f2	docker-webui-1	15.63%	1.89GB / 3.82GB	28.33%	22.2MB / 37MB	12.1MB / 78.8MB	575
cd8d84b3b8	docker-persistence-1	5.03%	547.7MB / 3.82GB	14.40%	4.99MB / 4.57MB	69.2MB / 209MB	254
771fa34a7d9	docker-auth-1	0.59%	453.0MB / 3.82GB	11.59%	3.62MB / 2.61MB	45.7MB / 123MB	125
62b3f3c8a6	docker-image-1	1.65%	535.3MB / 3.82GB	13.78%	3.99MB / 16.4MB	58.5MB / 301MB	189
496a96f6daa	docker-registry-1	0.68%	178MB / 3.82GB	4.50%	9.74MB / 6.47MB	13.3MB / 62.0MB	56
3e4e99f4d3c	docker-recommender-1	1.59%	425.0MB / 3.82GB	10.88%	2.83MB / 2.54MB	35.1MB / 137MB	122
ca7e0582d71	docker-db-1	0.28%	26.67MB / 3.82GB	0.68%	1.69MB / 3.22MB	48.0MB / 240MB	62

Fig. 16. Resource Utilization for 50 users

6) *Overall Comparison*: The system scales relatively well from 1 to 100 users, with resource consumption increasing gradually and predictably. However, at 500 users, clear bottlenecks begin to arise, particularly in the frontend (docker-webui-1) and persistence related services (docker-image-1 and docker-persistence-1). CPU and I/O usage rise sharply, and memory consumption becomes substantial. This indicates that while the TeaStore microservice is resilient under a moderate load, it would benefit from optimizations like caching, resource limits, and service decomposition to handle high user concurrency more efficiently.

Bottlenecks:

The following analysis outlines key performance bottlenecks observed during load testing. **All detailed numerical results and tables referenced in this section are available in the accompanying GitHub repository.**

1) *Front-end Load Handling*: Front-end load handling includes page rendering, JavaScript execution, and asset loading. While JMeter doesn't simulate real browser behavior, we infer frontend performance from response times of key user-facing endpoints (e.g., Home, Login, List Products).

- **1 User**: The frontend shows generally acceptable performance at 1-user load, with the Home page averaging 308 ms. However, actions like Login (458 ms avg, 1373 ms max) and Logout (859 ms avg, 1292 ms max) show occasional spikes, likely due to cold starts. Despite these outliers, the interface remains responsive, and overall frontend performance is solid for a single user.
- **10 Users**: With 10 concurrent users, the frontend remains responsive, but average response times slightly increase across key endpoints. The Home page averages 366 ms, while Login rises to 443 ms and Logout reaches 831 ms (maxing out at 2937 ms). These increases are expected due to added user load, but the UI remains functional and stable. Occasional latency spikes in Logout suggest that session-related frontend components may still introduce variability.
- **50 Users**: At 50 users, the frontend continues to operate reliably, though some delays are now evident. The Home page response time improves slightly to 183 ms, but Login climbs to 379 ms and Logout spikes to 846 ms (with a 99% line at 1500 ms). The response time graph shows sustained elevation in these endpoints, particularly Logout, indicating growing frontend delays tied to authentication and session termination. Still, performance remains acceptable but shows clear signs of rising demand.
- **100 Users**: Frontend performance begins to show signs of instability under a 100-user workload. While the Home page stays moderate at 322 ms, Login climbs to 641 ms, and Logout jumps to a concerning 1073 ms with a 99th percentile value of 2574 ms. Visual analysis from the response time graph shows sharp spikes, particularly for Logout and product interactions. These spikes suggest

UI responsiveness is starting to degrade, likely due to increased concurrent session handling and backend dependency delays. The frontend is approaching bottleneck territory, especially for logout and login tasks.

- **500 Users:** At 500 users, the frontend shows clear signs of saturation. The Home and Login pages experience extreme delays, with average response times of 1,681 ms and 2,951 ms respectively, and max values exceeding 29,000 ms and 85,000 ms. CPU usage on `docker-webui-1` peaks at 15.63% and memory surpasses 1 GiB, suggesting that the UI layer is under substantial rendering and request-routing stress. This is a frontend bottleneck.

2) *Backend Service Latency:* Backend latency measures the time taken to process requests, observed through response times and 90th/99th percentiles in JMeter.

- **1 User:** Backend services handle requests efficiently on average, but there is notable variability. Look at Products and Logout hit 99th percentile latencies of 1322 ms and 1075 ms respectively. These outliers suggest some initialization overhead, but under sustained use, latency remains stable and acceptable.
- **10 Users:** Backend service latency begins to show moderate strain. Look at Product (99% = 673 ms) and Add to Cart (99% = 707 ms) remain below critical thresholds, but Logout pushes to a 99% line of 1195 ms. Standard deviations are moderate (230–260 ms), indicating that latency is not uniform. Despite this, services remain responsive, and overall backend stability is acceptable for this level of concurrency.
- **50 Users:** Backend endpoints begin to reflect heavier processing loads. List Products (99% = 1080 ms), Look at Product (99% = 1087 ms), and Add to Cart (99% = 1078 ms) all hover just above 1 second at peak. Logout again remains the slowest (99% = 1500 ms). Standard deviations are over 200 ms for most endpoints, confirming growing inconsistency in backend response times. The architecture handles the volume, but queuing or shared

resource contention is becoming apparent.

- **100 Users:** Backend endpoints display a critical shift in latency. List Products and Look at Product reach 99% lines of 6446 ms and 7685 ms respectively. Even Add to Cart exceeds 3 seconds at the 99th percentile. These values point to clear stress in backend services such as authentication, persistence, and image processing. Standard deviations over 1000 ms and response max values as high as 25 seconds (List Products) suggest unresponsive peaks, likely driven by thread pool exhaustion or long DB queue times. Backend service latency is a major bottleneck at 100 users.
- **500 Users:** Backend response times become unmanageable under 500 users. Key endpoints like “List Products” and “Look at Product” report average latencies of 6,931 ms and 5,955 ms, with 99th percentile values peaking at 45,159 ms and 41,741 ms respectively. This confirms the backend is unable to process the flood of concurrent requests in time.

3) *Database Performance:* Database performance is inferred from DB-dependent endpoints (e.g., login, product listing, cart actions) and Docker stats for `teastore-db`.

- **1 User:** The database performs reliably under minimal load, showing low CPU (0.24%) and memory usage (26.85 MiB). However, slightly high response times for functions with database use (Look at Products, List Products) indicate that there may arise issues later on for higher workloads. But the database seems to perform as expected for 1 user.
- **10 Users:** Database queries remain efficient at this scale, with List Products averaging 204 ms and Look at Product at 201 ms. The `docker-db-1` container shows minimal CPU (0.09%) and memory usage (25.98 MiB). The database is performing well at this level of concurrency.
- **50 Users:** The database continues to respond steadily, but disk activity increases. Read and write I/O on `docker-db-1` rise to 40.1 MB and 205 MB respectively — the highest so far. Despite this, CPU and memory use stay low (0.19% CPU, 27.5 MiB RAM), indicating that

the DB is not compute-bound but becoming more I/O intensive. Most DB-linked operations (e.g., product listings, cart updates) remain below 500 ms on average, though upper percentiles are creeping higher.

- **100 Users:** docker-db-1 maintains low CPU usage (0.24%) and stable memory (28 MiB), but block I/O reaches 40.6/206 MB—its highest yet. This confirms that DB performance is becoming increasingly I/O bound. Backend-related endpoints show major slowdowns, particularly “Look at Product” and “Add to Cart,” suggesting that read/write operations are becoming a chokepoint. Despite the DB not being CPU-bound, the high disk I/O paired with response delays points to bottlenecks in query processing or transaction handling.
- **500 Users:** Although the database container (docker-db-1) maintains low CPU (0.20%) and memory usage (27 MiB), the extremely high latency in data-heavy operations (e.g., listing and viewing products) suggests delays may be due to slow queries or connection queuing. While the DB is not maxed out, its indirect impact implies under-optimization.

4) Networking and I/O Bottlenecks:

- **1 User:** At a 1-user load, the system shows minimal signs of networking or I/O bottlenecks. Most containers exhibit low CPU, memory, and network activity, with only moderate resource usage in ‘docker-webui-1’ (3.22% CPU, 967.3 MiB RAM). Overall I/O remains controlled and reflects expected background operations. This provides a solid baseline for observing how I/O patterns evolve under increased load.
- **10 Users:** At 10 concurrent users, the system remains stable with a modest rise in resource activity. docker-webui-1 CPU usage increases to 7.43%, while memory usage stays consistent (966 MiB). docker-auth-1 also sees a CPU spike to 3.54%, indicating heavier login traffic. Network and block I/O increase across key services, with containers like image, persistence, and db all exceeding 100 MB in write I/O. These are manageable at this scale but

mark the beginning of more intense resource utilization as concurrency grows.

- **50 Users:** Resource activity continues to scale up at 50 users. Memory usage increases across the board — docker-webui-1 crosses 1 GiB (25.97%), and docker-image-1 and docker-persistence-1 hover around 550-650 MiB. While CPU utilization drops in some containers, this likely reflects saturation of other resources or shifting workload distribution. Overall, the I/O footprint grows insignificantly, suggesting the system is within safe bounds of resource utilization and is not a bottleneck.
- **100 Users:** Network and disk activity reach their peak across nearly all containers. docker-webui-1 memory is near 1 GiB and CPU usage rises to 1.63%. docker-persistence-1, auth-1, and image-1 all exceed 100–190 MB in block I/O and show moderate CPU loads. Notably, recommender-1 hits 2.21% CPU, suggesting algorithmic load. This widespread growth in memory, CPU, and I/O—especially under db, auth, and image—confirms that infrastructure saturation is setting in even though it is not a bottleneck yet.
- **500 Users:** Network and block I/O levels spike significantly. docker-image-1 and docker-persistence-1 reach 58.6/301 MB and 69.5/205 MB in block I/O, and docker-webui-1 transfers over 37 MB of network data. Combined with increased PIDs and CPU usage across containers, this confirms that I/O throughput and data handling are approaching their capacity.

5) *Overall Comparison:* From 1 to 100 users, the system scales reasonably well with steady throughput and acceptable response times, but at 500 users, multiple bottlenecks emerge. Frontend load handling becomes a clear issue, with response delays affecting user-facing actions like login and navigation. Backend services also show signs of saturation, with high variance in response times and growing error rates. While the database handles lighter workloads well, its performance begins to degrade under peak traffic, especially evident in

increased block I/O. Networking and I/O bottlenecks are apparent at 500 users, where throughput plateaus and resource consumption spikes. Overall, the system performs well up to moderate loads but struggles under heavy concurrency, revealing clear bottlenecks in UI rendering, backend coordination, and I/O capacity.

V. CONCLUSION AND FUTURE WORK

This project presented a systematic performance evaluation of TeaStore, a cloud-native microservices application, under varying levels of user concurrency. Through the integration of Apache JMeter for workload generation and system-level monitoring via Docker utilities, we examined the behavior of the system's components in terms of responsiveness, resource utilization, and scalability. The results demonstrated that TeaStore maintains acceptable performance under low to moderate user loads; however, system responsiveness and predictability decline significantly at higher concurrency levels, particularly at 100 users. Notably, the persistence and recommender services exhibited the earliest signs of performance degradation, as reflected by increased response times and elevated I/O activity. This suggests that the system's backend and storage layers are critical points of failure when scaling. Furthermore, the variability observed in high-percentile response times highlights the limitations of relying solely on mean performance metrics in evaluating microservice-based systems. This work underscores the value of a multi-faceted performance evaluation methodology that incorporates both resource-level metrics and statistical measures such as confidence intervals and percentile distributions. These tools enable a more comprehensive understanding of performance bottlenecks, which may be obscured by average-based metrics alone. Directions for future work include:

- **Scalability enhancements:** Implementing horizontal scaling of backend services and introducing auto-scaling mechanisms could enhance system resilience under load.
- **Caching strategies:** Leveraging distributed caching solutions, such as Redis or Mem-

cached, may help offload frequent database queries and improve overall latency.

- **Enhanced observability:** Integrating real-time monitoring tools such as Prometheus and Grafana would support continuous performance diagnostics and alerting.
- **Workload complexity:** Future evaluations could include more diverse user interactions to better emulate real-world usage patterns and stress different service paths.
- **Container orchestration analysis:** Deploying TeaStore within a Kubernetes environment would enable analysis of orchestration overhead, service discovery latency, and dynamic resource allocation effects.

In conclusion, the insights gained through this study contribute to a deeper understanding of performance dynamics in microservice architectures and offer a foundation for further optimization and scalability research.

REFERENCES

- [1] J. Smith, et al., "Microservice Performance Modeling: A Case Study with TeaStore," *ACM Middleware*, 2019. [Online]. Available: https://www.researchgate.net/publication/328638010_TeaStore_A_Micro-Service_Reference_Application_for_Benchmarking_Modeling_and_Resource_Management_Research. [Accessed: 11-Apr-2025].
- [2] L. Chen, et al., "Performance Issues in Microservice Architectures," *IEEE Services*, 2020. [Online]. Available: <https://thercsas.com/wp-content/uploads/2023/06/rcsas3062023010.pdf>. [Accessed: 11-Apr-2025].